达梦数据库

dmfldr 使用手册

达梦技术手册

前言

概述

本文档主要介绍如何使用 dmfldr 快速装载工具进行 DM 数据库中表数据的快速载入和导出。

读者对象

本文档主要适用于 DM 数据库的：

- 开发工程师
- 测试工程师
- 技术支持工程师
- 数据库管理员

通用约定

在本文档中可能出现下列标志，它们所代表的含义如下：

表 0.1 标志含义

标志	说明
 警告：	表示可能导致系统损坏、数据丢失或不可预知的结果。
 注意：	表示可能导致性能降低、服务不可用。
 小窍门：	可以帮助您解决某个问题或节省您的时间。
 说明：	表示正文的附加信息，是对正文的强调和补充。

在本文档中可能出现下列格式，它们所代表的含义如下：

表 0.2 格式含义

格式	说明
宋体	表示正文。
Courier new	表示代码或者屏幕显示内容。
粗体	表示命令行中的关键字（命令中保持不变、必须照输的部分）或者正文中强调的内容。标题、警告、注意、小窍门、说明等内容均采用粗体。
<>	语法符号中，表示一个语法对象。
::=	语法符号中，表示定义符，用来定义一个语法对象。定义符左边为语法对象，右边为相应的语法描述。
|	语法符号中，表示或者符，限定的语法选项在实际语句中只能出现一个。
{ }	语法符号中，大括号内的语法选项在实际的语句中可以出现 0…N 次(N 为大于 0 的自然数)，但是大括号本身不能出现在语句中。
[ ]	语法符号中，中括号内的语法选项在实际的语句中可以出现 0…1 次，但是中括号本身不能出现在语句中。
关键字	关键字在 DM_SQL 语言中具有特殊意义，在 SQL 语法描述中，关键字以大写形式出现。但在实际书写 SQL 语句时，关键字既可以大写也可以小写。

访问相关文档

如果您安装了 DM 数据库，可在安装目录的“\doc”子目录中找到 DM 数据库的各种手册与技术丛书。

您也可以通过访问我们的网站 www.dameng.com阅读或下载 DM的各种相关文档。

联系我们

如果您有任何疑问或是想了解达梦数据库的最新动态消息，请联系我们：

网址：www.dameng.com

技术服务电话：400-991-6599

技术服务邮箱：dmtech@dameng.com

目录

1 概述 1

1.1 功能简介 1

1.2 系统结构 1

2 DMFLDR 入门 3

2.1 启动 DMFLDR 3

2.2 查看 DMFLDR 参数 3

2.3 DMFLDR 参数简介 5

3 DMFLDR 实战 14

3.1 DMFLDR 控制文件 14

3.2 指定数据文件 18

3.2.1 在控制文件中指定数据文件 18

3.2.2 使用 DATA 参数指定数据文件 19

3.3 数据转换与错误数据文件 20

3.4 服务器端错误数据处理 21

3.5 大字段数据处理 22

3.5.1 大字段数据的导出 22

3.5.2 DIRECT 为 TRUE 时大字段数据的载入 23

3.5.3 DIRECT 为 FALSE 时大字段数据的载入 24

3.6 日志文件及日志信息 26

3.7 自增列装载 28

3.8 数据排序 30

3.9 空值处理 31

3.10 类类型装载 33

3.11 个性化设置 36

3.12 主备切换时的数据继续载入 38

3.13 MPP 本地分发 38

3.14 提升 DMFLDR 性能 40

3.15 DMFLDR 使用限制 41

1 概述

1.1 功能简介

dmfldr（DM Fast Loader）是 DM7 提供的快速数据装载命令行工具。用户通过使用 dmfldr 工具能够把按照一定格式排序的文本数据以简单、快速、高效的方式载入到 DM 数据库中，或把 DM 数据库中的数据按照一定格式写入文本文件。

1.2 系统结构

dmfldr 的系统结构如图 1.1 所示。

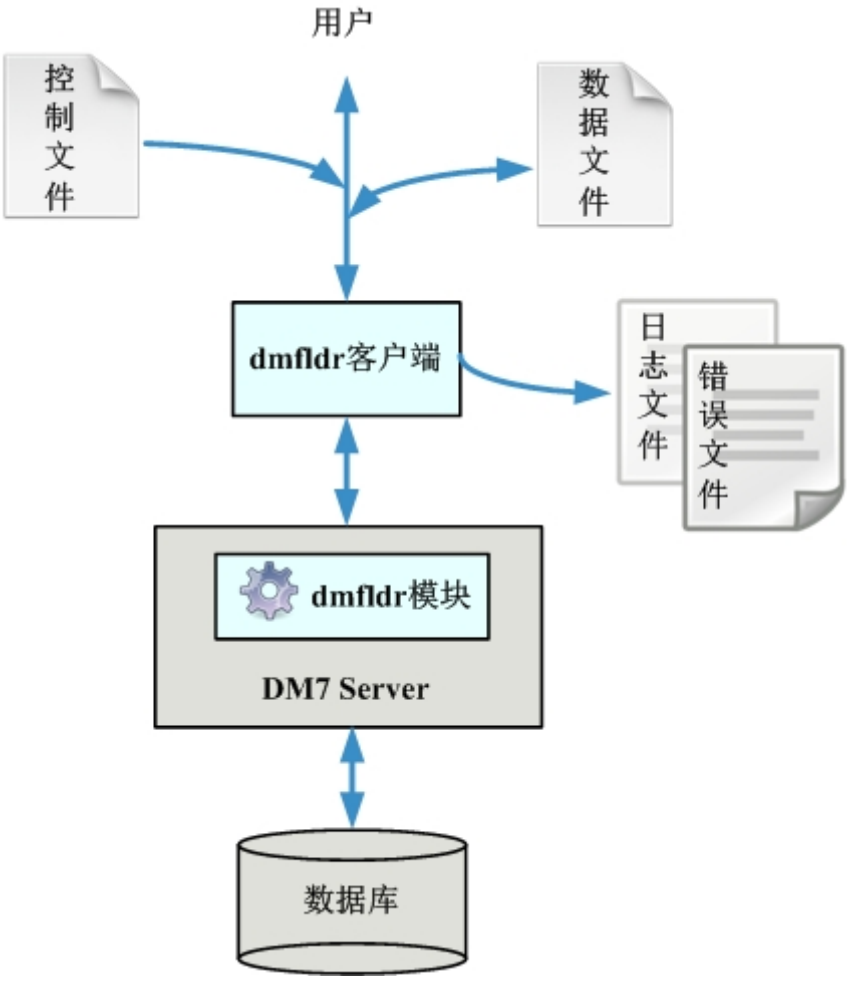


图 1.1 dmfldr 结构示意图

如图所示，dmfldr 实际上除了客户端工具，还包含一个在 DM7 数据库服务器中的 dmfldr 功能模块，它们共同完成 dmfldr 的各项功能。

当进行数据载入时，dmfldr 客户端接收用户提交的命令与参数，分析控制文件与数据文件，将数据打包发送给服务器端的 dmfldr 模块，由服务器完成数据的真正装载工作。并分析服务器返回的消息，必要时根据用户参数指定生成日志文件与错误数据文件。

当进行数据导出时，dmfldr 客户端接收用户提交的命令与参数，分析控制文件，将用户要求转换成相应消息发送给服务器端的 dmfldr 模块。服务器解析并打包需要导出的数据，发送给 dmfldr 客户端，客户端将数据写入指定的数据文件，必要时根据用户参数指定生成日志文件。

2 dmfldr 入门

本章简单介绍如何启动 dmfldr 和 dmfldr 支持的参数简介。通过阅读本章，读者可以了解 dmfldr 通过各参数能提供的各项功能，不过要想熟练灵活地使用 dmfldr 还需要继续阅读下一章。

2.1 启动 dmfldr

安装好 DM7 数据库管理系统后，在安装目录的“bin”子目录下可找到 dmfldr 执行文件。

启动操作系统的命令行窗口，进入“dmfldr”所在目录，可以准备启动 dmfldr 工具了。

dmfldr 的使用必须指定必要的参数，否则工具会报错“无效的参数个数”并退出。为 dmfldr 指定参数的格式为：

```
dmfldr keyword=value [keyword=value ...]
```

例如：

```
dmfldr USERID=SYSDBA/SYSDBA CONTROL='c:\fldr.ctl'
```

如例子所示，USERID 和 CONTROL 是启动 dmfldr 必须要指定的参数，且 USERID 必须是第一个参数，CONTROL 必须是第二个参数。

2.2 查看 dmfldr 参数

dmfldr 使用较为灵活，参数较多，用户可以使用“dmfldr help”查看各参数的简单信息。

```
dmfldr help

格式: DMFLDR   KEYWORD=value

例程: DMFLDR   SYSDBA/SYSDBA CONTROL='c:\fldr.ctl'

USERID 必须是命令行中的第一个参数

CONTROL 必须是命令行中的第二个参数
```

字符串类型参数必须以引号封闭

关键字	说明（默认）
USERID	用户名/口令 格式:USER/PWD@SERVER:PORT#SSL_PATH@SSL_PWD
CONTROL	控制文件，字符串类型
LOG	日志文件，字符串类型 (dfldr.log)
BADFILE	错误数据记录文件，字符串类型 (dfldr.bad)
SKIP	初始忽略逻辑行数 (0)
LOAD	需要装载的行数 (ALL)
ROWS	提交频次 (50000), DIRECT 为 FALSE 有效
DIRECT	是否使用快速方式装载 (TRUE)
SET_IDENTITY	是否插入自增列 (FALSE)
SORTED	数据是否已按照聚集索引排序 (FALSE)
INDEX_OPTION	索引选项 (1) 1 不刷新二级索引，数据按照索引先排序，装载完后再 将排序的数据插入索引 2 不刷新二级索引，数据装载完成后重建所有二级索引
ERRORS	允许的最大数据错误数 (100)
CHARACTER_CODE	字符编码，字符串类型 (GBK, UTF-8, SINGLE_BYTE, EUC-KR)
MODE	装载方式，字符串类型 IN 表示载入，OUT 表示载出， OUTORA 表示载出 ORACLE (IN)
CLIENT_LOB	大字段目录是否在本地 (FALSE)
LOB_DIRECTORY	大字段数据文件存放目录
LOB_FILE_NAME	大字段数据文件名称，仅导出有效(默认 dmfldr.lob)
BUFFER_NODE_SIZE	读入文件缓冲区的大小(10),有效值范围 1~2048
READ_ROWS	工作线程一次最大处理的行数，最大支持 2^26-10000， 与 BUFFER_NODE_SIZE 取较小值 (100000)
NULL_MODE	载入时 NULL 字符处是否处理为 NULL 载出时空值是否处理为 NULL 字符串 (FALSE)

SEND_NODE_NUMBER	运行时发送节点的个数，有效值范围 16~65535
TASK_THREAD_NUMBER	处理用户数据的线程数目，有效值范围 1~128
BLDR_NUM	服务器 BLDR 数目，默认 64,有效值范围 1~65535
BDTA_SIZE	bdta 的大小，默认 5000，有效值范围 100~10000
COMPRESS_FLAG	是否压缩 bdta (FALSE)
MPP_CLIENT	mpp 环境，是否本地分发(TRUE)
BLOB_TYPE	BLOB 类型字段数据值的实际类型，字符串类型 (HEX_CHAR) HEX 表示值为十六进制，HEX_CHAR 表示值为十六进制字符类型
OCI_DIRECTORY	OCI 动态库所在的目录
DATA	指定数据文件路径
ENABLE_CLASS_TYPE	允许用户导入 CLASS 类型数据 (默认 FALSE)
HELP	打印帮助信息

2.3 dmfldr 参数简介



注意： **USERID必须是第一个参数，CONTROL必须是第二个参数，这两个参数都不能省略；其余参数均为可选参数，可以不指定，指定时也无顺序要求**

- **USERID**

用于指定登录数据库的信息，包含用户名、密码、服务器地址、端口号和安全证书存放路径。参数值格式为：

```
用户名/密码[@主库地址[:端口号][#安全证书存放路径@安全证书密码]]
```

如果用户使用默认端口号 5236 登录则端口号可以不设置；如果登录的是本地的服务器则主库地址可以省略；如果不使用通信加密则安全证书相关设置可以省略。

此参数为必选参数且必须位于参数位置的第一个。

- **CONTROL**

控制文件的路径，字符串类型。控制文件用于指定数据文件中数据的格式，在数据载入时，dmfldr 根据控制文件指定的格式来解析数据文件；导出数据时，dmfldr 也会根据控

制文件指定的列分隔符、行分隔符等生成数据文件。控制文件中还可以指定其他的一些 dmfldr 参数值。对控制文件格式的详细介绍见下一章。

此参数为必选参数且必须位于参数位置的第二个。

- **LOG**

dmfldr 的日志文件路径，字符串类型。默认日志文件名为 fldr.log。日志文件记录了 dmfldr 运行过程中的工作信息、错误信息以及统计信息。

用户也可以通过设置控制文件中的 OPTION 选项来指定日志路径，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

此参数为可选参数。

- **BADFILE**

记录错误数据的文件路径，字符串类型。默认的错误文件名为 fldr.bad。文件记录的信息为数据文件中存在格式错误的行数据以及转换出错的行数据。

用户也可以通过设置控制文件中的 OPTION 选项来指定错误数据文件的路径，同时也可以在控制文件的 LOAD 节点中指定错误数据文件的路径。BADFILE 选项的最终值的优先选择顺序为 LOAD 节点选项，OPTION 选项，参数选项，用户可以同时对三种设置方式中的一个或多个设置，但最终的值只取一个。

此参数为可选参数，作用于 MODE 为 IN 的情况下，当 MODE 为 OUT 时无效。

- **SKIP**

跳过数据文件起始的逻辑行数，整形数值。默认的跳过起始行数为 0 行。如果用户指定了多个文件，且起始文件中的行数不足 SKIP 所指定的行数，则 dmfldr 工具会扫描下一个文件直至累加的行数等于 SKIP 所设置的行数或者所有文件都已扫描结束。

用户也可以通过设置控制文件中的 OPTION 选项来指定 SKIP 参数，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

- **LOAD**

装载的最大行数，整形数值。默认的最大装载行数为数据文件中的所有行数。LOAD 指定的值是不包括 SKIP 指定的跳过的行数。

用户也可以通过设置控制文件中的 OPTION 选项来设置 LOAD 的值，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

- **ROWS**

每次提交的行数，整形数值。默认的提交行数为 50000 行。提交行数的值表示提交到服务器的行数，并不一定代表按照数据文件中的数据顺序的行数。用户可以根据实际情况调整每次提交的行数，以达到性能的最佳点。

用户也可以通过设置控制文件中的 OPTION 选项来设置 ROWS 值，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，作用于 MODE 为 IN 且 DIRECT 为 FALSE 的情况下，其他情况下无效。

- **DIRECT**

数据装载方式，布尔值。默认为 TRUE。DIRECT 指定了装载的方式，当为 TRUE 时，dmfldr 选择快速的载入模式，通过数据的转换和数据的封装直接对 B 树进行操作，省去了普通插入方式下各个操作符之间的跳转，提升了装载的效率，但对于约束的检查等由用户保证，dmfldr 将不处理有约束冲突的数据。当为 FALSE 时，dmfldr 选择普通的插入方式装载数据，可以保证数据的正确性和约束的有效性，效率比前者要低。

用户也可以通过设置控制文件中的 OPTION 选项来设置 DIRECT 值，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，作用于 MODE 为 IN 的情况下，当 MODE 为 OUT 时无效。

- **SET_IDENTITY**

设置自增列选项，布尔值。默认为 FALSE。如果指定 SET_IDENTITY 选项值为 TRUE，则 dmfldr 将把从数据文件中读取的自增列值作为目标值插入数据库表中，用户应当保证每一行的自增列的值符合自增列的规则。如果 SET_IDENTITY 选项值设置为 FALSE，则 dmfldr 将忽略数据文件中对应自增列的值，服务器将自动生成自增列的值插入每一行的对应列。

用户也可以通过设置控制文件中的 OPTION 选项来设置 SET_IDENTITY 的值，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，作用于 MODE 为 IN 的情况下，当 MODE 为 OUT 时无效。

- **SORTED**

数据是否已经按照聚集索引排序，布尔值。默认为 FALSE。如果设置为 TRUE，则用户必须保证数据已按照聚集索引排序完成，并且如果表中存在数据，需要插入的数据索引值要比表中数据的索引值大，服务器在做插入操作时顺序进行插入。如果设置为 FALSE，则服务器对于每条记录进行定位插入。

用户也可以通过设置控制文件中的 OPTION 选项来设置 SORTED 的值，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，作用于 MODE 为 IN 且 DIRECT 为 TRUE 的情况下，对于其他情况此参数无效。

- **INDEX_OPTION**

索引的设置选项，整形数值。默认为 1。INDEX_OPTION 的可选项有 1 和 2。1 代表服务器装载数据时先不刷新二级索引，而是将新数据按照索引预先排序，在装载完成后，再将排好序的数据插入索引；2 代表服务器在快速装载过程中不刷新二级索引数据，只在装载完成时重建所有二级索引。

用户也可以通过设置控制文件中的 OPTION 选项来设置 INDEX_OPTION 的值，参数值的优先选择顺序为线 OPTION 选项，后参数选项。

此参数为可选参数，作用于 MODE 为 IN 且 DIRECT 为 TRUE 的情况下，对于其他情况此参数无效。

- **ERRORS**

最大的容错个数，整形数值。默认为 100。当 dmfldr 在插入过程中出现错误的个数超过了 ERRORS 所设置的数目，则 dmfldr 会停止载入，当前时间点的所有正确数据将会被提交到服务器。如果载入过程中不允许出现错误则可以设置 ERRORS 为 0，如果允许所有的错误出现，则可以设置 ERRORS 为一个非常大的数。ERRORS 所统计的错误包含在数据转换和数据插入过程中所产生的数据错误。

用户也可以通过设置控制文件中的 OPTION 选项来设置 ERRORS 的值，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，作用于 MODE 为 IN 的情况下，当 MODE 为 OUT 时无效。

- **CHARACTER_CODE**

数据文件中数据的编码格式，字符串类型。默认为 GBK。CHARACTER_CODE 的可选项有 GBK，UTF-8，SINGLE_BYTE 和 EUC-KR 四种：GBK 对应中文编码；UTF-8 对应 UTF-8 国际编码；SINGLE_BYTE 对应单字节字符编码；EUC-KR 对应韩文字符集。

在 MODE 为 IN 的情况下，指定编码格式为 SINGLE_BYTE 时，dmfldr 将不做字符完整性检查，按单字节顺序读取每个字符。指定编码格式为 GBK 或者 UTF-8 时，dmfldr 将对每一个字符做字符的完整性检查，确保数据的正确性。

用户在使用 dmfldr 时可以根据不同的数据文件调整编码的格式确保装载的正确性，同时如果可以确保数据文件中的数据为单字节字符，则指定 SINGLE_BYTE 的载入效率将优于指定 GBK 和 UTF-8 的情况。

当 MODE 为非 IN 时，不支持 SINGLE_BYTE。

用户也可以通过指定控制文件中的 OPTION 选项来设置 CHARACTER_CODE 的值，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

- **MODE**

dmfldr 的装载模式，字符串类型。默认值为 IN。MODE 的可选项有 IN、OUT 和 OUTORA 三种。

IN 模式指从数据文件中将数据装载入数据库，这种模式下控制文件的格式对应为数据文件中现有数据的格式；OUT 模式指从数据库中将数据导出到数据文件，这种模式下控制文件所指定的格式为数据存放在数据文件中的格式。需要说明的是在 OUT 模式下，如果指定了多个数据文件，则 dmfldr 最终只会将数据写入第一个数据文件，如果数据文件存在，dmfldr 会直接将数据导入；如果数据文件不存在，会先创建文件后导入。

OUT 模式下的控制文件与 IN 模式下的控制文件格式相同，用户可以通过使用同一个文件进行载入和导出数据。

OUTORA 模式表示导出 ORACLE 表的数据，此模式下暂不支持带有大字段表的导出。

用户也可以通过指定控制文件中的 OPTION 选项来设置 MODE 的值，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

- **CLIENT_LOB**

指明 LOB_DIRECTORY 表示的目录是否是客户端本地目录，布尔类型，默认值为 FALSE。

CLIENT_LOB 仅在 MODE 为 IN 且 DIRECT 为 TRUE 时有效，指明在载入大字段对象数据时，LOB_DIRECTORY 参数指定的目录是否是客户端本地目录。若为 TRUE，表示目录为客户端本地目录；若为 FALSE，表示目录为 DM 服务器所在主库的目录。

- **LOB_DIRECTORY**

指明 dmfldr 使用的大字段数据存放的目录，字符串类型。

当 MODE 为 OUT 时，dmfldr 生成大字段对应的数据文件，文件名由 LOB_FILE_NAME 指定，并存放于 LOB_DIRECTORY 指定的目录，如果未指定

LOB_DIRECTORY 则存放于指定的导出数据文件同一目录。对于 MODE 为 OUT 的情况，此参数为可选参数。

当 MODE 为 IN 且 DIRECT 为 TRUE 时，此时数据载入若涉及到大字段对象，需要用户指定大字段数据文件。若 CLIENT_LOB 为 TRUE，LOB_DIRECTORY 应指定大字段数据文件所在的客户端本地目录；若 CLIENT_LOB 为 FALSE，用户必须先把相关文件传送到 DM 服务器所在主库，然后使用 LOB_DIRECTORY 指明存放目录。此时此参数为必选参数。

当 MODE 为 IN 且 DIRECT 为 FALSE 时，此参数无效。

用户也可以通过指定控制文件中的 OPTION 选项来设置 LOB_DIRECTORY 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

- **LOB_FILE_NAME**

指明 dmfldr 导出大字段数据的文件名，字符串类型，默认为 dmfldr.lob。

当 MODE 为 OUT 时，dmfldr 生成大字段对应的数据文件名由 LOB_FILE_NAME 指定，若未指定默认为 dmfldr.lob，文件存放于 LOB_DIRECTORY 指定的目录。

用户也可以通过设置控制文件中的 OPTION 选项来设置 LOB_FILE_NAME 的值，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，作用于 MODE 为 OUT 的情况下，当 MODE 为 IN 时无效。

- **BLOB_TYPE**

指定 BLOB 数据值的实际类型，字符串类型。可选项为 HEX 表示值为十六进制，HEX_CHAR 表示值为十六进制字符，默认为 HEX_CHAR。

用户也可以通过设置控制文件中的 OPTION 选项来设置 BLOB_TYPE 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，只在 DIRECT 参数为 FALSE 的情况下有效。

- **BUFFER_NODE_SIZE**

指定读取文件缓冲区页大小，整数类型，单位为 M，范围为 1~2048，默认为 10。

BUFFER_NODE_SIZE 的值越大，缓冲区的页越大，每次读取的数据就越多，每次发送到服务器的数据也就越多，效率越高。但其大小受 dmfldr 客户端内存大小限制。

用户也可以通过设置控制文件中的 OPTION 选项来设置 BUFFER_NODE_SIZE 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

- **READ_ROWS**

指定读取缓冲区每次读取的最大行数，整数类型，范围为 0~(2^{26}-10000)默认为 100000。

在某些情况下，1M 的 BUFFER_NODE_SIZE 读入的数据行数很大，而后续操作处理不了这么大的行数，此时可以用 READ_ROWS 来限制处理的行数。dmfldr 取 READ_ROWS 和 BUFFER_NODE_SIZE 中较小的值作为一次处理的行数。

用户也可以通过设置控制文件中的 OPTION 选项来设置 READ_ROWS 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

- **NULL_MODE**

指定载入和导出数据时对 NULL 字符串和空值的处理方式，布尔类型，默认为 FALSE。

若 NULL_MODE 为 TRUE，数据载入时将 NULL 字符串处理为空值，数据导出时则空值处理为 NULL 字符串；若设置为 FALSE，数据载入时将 NULL 字符串处理为字符串，数据导出时将空值处理为空串。

用户也可以通过设置控制文件中的 OPTION 选项来设置 NULL_MODE 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

- **SEND_NODE_NUMBER**

指定 dmfldr 在数据载入时发送节点的个数，整数类型，范围为 16~65535，默认由系统计算一个初始值。若在数据载入时发现发送节点不够用，系统会动态增加分配。

用户也可以通过设置控制文件中的 OPTION 选项来设置 READ_ROWS 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，只在 MODE 为 IN 的情况下有效。

- **TASK_THREAD_NUMBER**

指定 dmfldr 在数据载入时处理用户数据的线程数目，整数类型，范围为 1~128。默认情况下，dmfldr 将该参数值设为系统 CPU 的个数，但当 CPU 个数大于 8 时，默认值都被置为 8。

用户也可以通过设置控制文件中的 OPTION 选项来设置 TASK_THREAD_NUMBER 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，只在 MODE 为 IN 的情况下有效。

- **BLDR_NUM**

水平分区表装载时，指定服务器 BLDR 的最大个数，整数类型，范围为 1～1024，默认为 100。

服务器的 BLDR 保存水平分区子表相关信息，BLDR_NUM 的设置也就指定了服务器能同时载入的水平分区子表的个数。

用户也可以通过设置控制文件中的 OPTION 选项来设置 BLDR_NUM 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，作用于 MODE 为 IN 的情况下，当 MODE 为 OUT 时无效。

- **BDTA_SIZE**

BDTA（Batch DaTA）的大小，整数类型，范围为 100~10000，默认为 5000。

BDTA 代表 DM7 数据库批量数据处理机制中一个批量，在内存、CPU 允许的条件下，增大 BDTA_SIZE 能加快装载速度；在网络是装载性能瓶颈时，增大 BADT_SIZE 影响不大。

用户也可以通过设置控制文件中的 OPTION 选项来设置 BADT_SIZE 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，作用于 MODE 为 IN 的情况下，当 MODE 为 OUT 时无效。

- **COMPRESS_FLAG**

指定是否压缩 BDTA，布尔类型，默认为 FALSE。

压缩 BDTA 能节省网络带宽，但同时也会加重 CPU 的负担，用户应根据具体应用情况考虑是否制定压缩。

用户也可以通过设置控制文件中的 OPTION 选项来设置 COMPRESS_FLAG 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

- **MPP_CLIENT**

指定当服务器环境为 MPP 环境时 dmfldr 的数据装载模式，布尔类型，默认为 TRUE。

当服务器环境为 MPP 环境时，若 MPP_CLIENT 为 TRUE，为客户端分发模式，数据在 dmfldr 客户端分发好后直接往指定站点发送数据；若 MPP_CLIENT 为 FALSE，为本地分发模式，dmfldr 客户端将数据全部发往连接的 MPP EP 站点。

MPP 环境下要配置 dmmal.ini 文件中的 MAL_INST_HOST 和 MAL_INST_PORT 参数。

用户也可以通过设置控制文件中的 OPTION 选项来设置 MPP_CLIENT 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

此参数只有当服务器环境为 MPP 环境时才有效。

- **OCI_DIRECTORY**

指定 ORACLE OCI 动态库所在的目录，字符串类型。

该参数与 MODE 参数配合使用，当 MODE 指定为 OUTORA 时，使用 OCI_DIRECTORY 指定 OCI 的动态库来构建导出环境。

用户也可以通过设置控制文件中的 OPTION 选项来设置 OCI_DIRECTORY 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

- **DATA**

指定数据文件路径，字符串类型。

一般情况下数据文件路径在控制文件中指定。DATA 参数值的优先顺序为先控制文件，后参数选项。如果控制文件中数据文件路径指定为‘*’，在命令行通过 DATA 参数指定数据文件路径，DATA 所指定的文件路径会替换‘*’。

- **ENABLE_CLASS_TYPE**

指定是否以 BLOB 方式载入或导出 CLASS 类型列数据，布尔类型，默认为 FALSE。

CLASS 类型是 DM 数据库服务器内部实现的数据类型，实际以 BLOB 类型存储。如果通过交互式工具或 DM 提供的接口等正常途径创建的 CLASS 类型数据，内部会转换成 BLOB 存储。而通过 dmfldr 直接导 CLASS 数据，没有进行转换，有可能出现载入的大对象数据无法转换成对应的 CLASS 类型。因此，当将 ENAME_CLASS_TYPE 设为 TRUE 时，用户要保证对应的 BLOB 数据能正确转换成对应的 CLASS 类型。

用户也可以通过设置控制文件中的 OPTION 选项来设置 ENAME_CLASS_TYPE 的值，参数值的优先顺序为先 OPTION 选项，后参数选项。

- **HELP**

获取帮助信息。

3 dmfldr 实战

在上一章简单了解了 dmfldr 的各参数含义后，本章介绍如何通过设置不同参数值使用 dmfldr 进行不同情况的数据装载。并介绍了如何通过设置参数提高 dmfldr 的性能以及使用限制。



说明：本章中皆以在 Linux 操作系统下进行操作为例，在 Windows 操作系统下目录分隔符与单引号转义的处理有些许区别，不再赘述

3.1 dmfldr 控制文件

控制文件 CONTROL 是启动 dmfldr 必须要指定的参数，用于指定数据文件中数据的格式。在数据载入时，dmfldr 根据控制文件指定的格式来解析数据文件；导出数据时，dmfldr 也会根据控制文件指定的列分隔符、行分隔符等生成数据文件。

控制文件中还可以指定其他 dmfldr 参数值。

dmfldr 控制文件的语法如下所示：

```
[OPTIONS(
<id>=<value>
……
       )]
LOAD [DATA]
INFILE [LIST] <path_name> [<row_term_option>] [,<path_name>
[<row_term_option>]]
[BADFILE <path_name>]
[APPEND|REPLACE|INSERT]
INTO TABLE [<schema>.]<tablename>
[FIELDS <delimiter>]
[<coldef_option>]
<id> ::=参数
```

```
<value> ::=值

<path_name> ::=文件地址

<path_name_list> ::=含文件列表文件地址

<row_term_option> ::=STR [X] ‘<value>’

<schema> ::=模式名

<tablename> ::=表名

<delimiter> ::= '<单字节字符>'

<coldef_option> ::=(<col_def>{ ,<col_def>})

<col_def > ::=<col_id> [<property_option>] [<fmt_option>] [<term_option>]

[<enclosed_option>][<constant_option>][<fun_option>]

<col_id> ::=列名

<property_option> ::=FILLER|<position_option>|NULL

<position_option> ::=position(p1:p2)

<fmt_option> ::=DATE FORMAT '<时间日期格式串>'

<term_option> ::= TERMINATED [BY] <wx_option>

<wx_option> ::= WHITESPACE|'<单字节字符>'

<enclosed_option> ::= [OPTIONALLY] ENCLOSE [BY] '<单字节字符>'

<constant> ::= CONSTANT "<常量>"

<fun_option> ::= "函数名称()"
```

对于上述控制文件语法，需要说明的是：

- dmfldr 在处理数据文件中换行符时 windows 默认为 0x0D0A(\r\n)，非 windows 默认为 0x0A(\n)，用户应该根据现有的数据文件中的换行符做相应的调整。对应选项为<row_term_option>，若指定的<value>值为二进制的字符串值需要指明[X]选项，<value>值不再需要以 0x 开头。若没有指明[X]选项，则<value>值为指定的字符串；
- 关于列分隔符，用户应当指定 FIELDS 或者 coldef_option 中的至少一种；
- 关于 LIST 选项，INFILE 使用 LIST 选项时，表明实际的数据文件路径存储在 INFILE 指定的文件中，该文件可以存储多个实际的数据文件路径，使用逗号或者换行分割；

- 关于 OPTIONS 选项，该选项支持命令行参数中除 userid，control，help 以外的所有参数的指定，每个参数值对使用空格或者换行分割。对于 option 中出现的参数，在 dmfldr 的指定执行参数中也出现的，dmfldr 会选择 option 中对应参数的值执行；
- 关于 property_option 选项
  - ✓ FILLER：跳过处理数据文件中指定列的值
  - ✓ position(p1:p2)：从数据文件中每行数据的第 p1 个字节到第 p2 个字节为该列值，包含边界 p1,p2。position 选项对大字段数据无效，若对大字段类型指定此选项会报错
  - ✓ NULL：指定的值为 NULL，忽略数据文件中的值

  property_option 参数仅对导入有效；
- 关于 term_option 选项，该选项用来指定数据文件中指定列的结束标志。列的结束标志可以是 WHITESPACE（空格）或者用户自定义的单字节字符。指定了 term_option 后，该列不需要用 FIELDS 分隔；
- 关于 enclosed_option 选项，在数据文件中，该列的前后由指定的单字节字符围绕；
- 关于 constant 选项，指定 constant 关键字后，数据文件中不需要为该列准备数据，如果指定了，该列数据将作为下一字段数据装载而导致数据混乱。

  constant 选项对大字段类型无效；
- 关于 fun_option 选项，目前只支持 trim()函数；

例：一个 dmfldr 控制文件的例子

```
OPTIONS
(
       SKIP = 0
       ROWS = 50000
       DIRECT = TRUE
       INDEX_OPTION = 2
)
LOAD DATA
```

```
INFILE '/opt/data/test1.txt' STR X '0D0A'
BADFILE '/opt/data/test1.bad'
INTO TABLE test1
FIELDS '|'
(
F1,
F2 DATE FORMAT 'YYYY-MM-DD',
F3 NULL,
F4 TERMINATED BY WHITESPACE ENCLOSE BY '(',
F5 CONSTANT "test",
F6 "trim()"
)

LOAD DATA
INFILE '/opt/data/test2.txt' STR X '0D0A'
BADFILE '/opt/data/test2.bad'
INTO TABLE test2
FIELDS '|'
(
C1 TERMINATED BY ' ',
C2,
C3 DATE FORMAT 'yyyy-mm-dd'
)
```

在这个例子中：

- OPTIONS 选项中定义了 SKIP、ROWS、DIRECT 和 INDEX_OPTION 参数
- 有两个 LOAD 选项，表明有一次装载同时处理两个表，在每个 LOAD 选项中：
  - 指定了数据文件和数据文件的换行符，还指定了 BADFILE 文件
  - 指定了操作的数据库基表、列分隔符以及列定义

3.2 指定数据文件

当 dmfldr 工作在 IN 模式时，从数据文件中读取数据并载入数据库；当工作在 OUT 模式时，从数据库中将指定数据导出到数据文件。

数据文件通常为文本文件，列与列之间由列分隔符隔开，行与行之间由行分隔符隔开。数据文件中的列分隔符和行分隔符由用户指定，并在控制文件中设置为与数据文件中的一致。

3.2.1 在控制文件中指定数据文件

可以在控制文件的 LOAD 节点中指定数据文件。

例如：

1) 建表 TEST

```
DROP TABLE TEST;

CREATE TABLE TEST(C1 INT,C2 INT,C3 DATE);
```

2) 编辑数据文件 test.txt，存放路径为/opt/data/test.txt，文件内容如下

```
1 1|2015-11-06

2 2|2015-11-05

3 3|2015-11_04
```

3) 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA

INFILE '/opt/data/test.txt'

INTO TABLE test

FIELDS '|'

(

C1 TERMINATED BY ' ',

C2,

C3 DATE FORMAT 'yyyy-mm-dd'

)
```

4) 使用 dmfldr 进行数据载入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
```

3.2.2 使用 DATA 参数指定数据文件

也可以使用 DATA 参数指定 dmfldr 的数据文件，数据文件路径的优先选择顺序为先控制文件，后参数选项。如果控制文件中数据文件路径指定为‘*’，在命令行通过 DATA 参数指定数据文件路径，DATA 所指定的文件路径会替换‘*’。

例如：

1） 建表 TEST

```
DROP TABLE TEST;

CREATE TABLE TEST(C1 INT,C2 INT,C3 DATE);
```

2） 编辑数据文件 test.txt，存放路径为/opt/data/test.txt，文件内容如下

```
1 1|2015-11-06

2 2|2015-11-05

3 3|2015-11_04
```

3） 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA

INFILE *

INTO TABLE test

FIELDS '|'

(

C1 TERMINATED BY ' ',

C2,

C3 DATE FORMAT 'yyyy-mm-dd'

)
```

4） 使用 dmfldr 进行数据载入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'

data=\'/opt/data/test.txt \'
```

3.3 数据转换与错误数据文件

dmfldr 使用的数据文件都是文本格式的，其中的列值都是以字符串的方式保存在数据文件中。要想将这些数据载入数据库表中，需要将字符串转换成数据库表各列对应的数据类型。dmfldr 支持所有 DM 数据库支持的列定义类型，包括字符串、数值、时间日期、时间日期间隔、大字段类型等。

若数据文件的编码方式与 DM 数据库服务器的编码方式不一样，dmfldr 还需要进行字符编码的转换。dmfldr 支持 UTF8 和 GBK 编码之间的相互转换。

数据类型和编码转换工作由 dmfldr 客户端进行，在这个过程中如果出现错误，dmfldr 会跳过该行继续后面的工作，并记录错误行到 BADFILE 指定的文件。常见的出错的情况有以下几种：

- 编码转换失败
- 目标列为字符串类型时，数据长度大于列定义长度
- 目标列为数值类型时，数据包含非法字符或者转换后超出该数值的范围
- 目标列为日期类型时，dmfldr 默认按 yyyy-mm-dd hh:mi:ss 的格式解析，如果数据不是这样的格式，需要指定对应列的时间日期 fmt 格式而未指定

dmfldr 错误数据的文件路径由 BADFILE 参数设置，默认的错误文件名为 fldr.bad。用户也可以通过设置控制文件中的 OPTION 选项来指定错误数据文件的路径，同时也可以在控制文件的 LOAD 节点中指定错误数据文件的路径。错误数据文件路径最终值的优先选择顺序为 LOAD 节点选项，OPTION 选项，参数选项。用户可以同时对三种设置方式中的一个或多个设置，但最终的值只取一个。BADFILE 仅作用于 dmfldr 的工作 MODE 为 IN 的情况下，MODE 为 OUT 时无效。

当数据类型和编码转换中存在错误数据，而错误数在允许的最大错误数范围内时，dmfldr 会将出错的数据记录到错误数据文件中。文件记录的信息为执行程序、时间、目标表、数据文件中存在格式错误的行数据以及转换出错的行数据。

允许的最大容错个数由 ERRORS 选项设置，默认为 100。当 dmfldr 客户端在数据类型和编码转换过程中出现的错误个数超过了 ERRORS 所设置的数目，dmfldr 会停止载入，当前时间点的所有正确数据将会被提交到服务器端。如果载入过程中不允许出现错误则可以将 ERRORS 设置为 0；如果允许所有的错误出现，则可以将 ERRORS 设置为一个非常大的数。

例如：

1） 建表 TEST

```
DROP TABLE TEST;

CREATE TABLE TEST(C1 INT,C2 INT,C3 DATE);
```

2） 编辑数据文件 test.txt，存放路径为/opt/data/test.txt，文件内容如下

```
1 1|2015-11-06

2 2|2015-11-05

3 3|2015-11_04

4 4|aaaa-bbb-ccc
```

3） 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA

INFILE '/opt/data/test.txt'

INTO TABLE test

FIELDS '|'

(

C1,

C2 TERMINATED BY ' ',

C3 DATE FORMAT 'yyyy-mm-dd'

)
```

4） 使用 dmfldr 进行数据载入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'

badfile=\'/opt/data/test.bad\'
```

5） 查看错误数据文件/opt/data/test.bad，其内容如下

```
dmfldr: 2015-11-09 16:56:52 SYSDBA->TEST 4|4 aaaa-bbb-ccc
```

3.4 服务器端错误数据处理

dmfldr 客户端将载入的数据进行数据转换和编码转换后，将转换正确的数据发往 DM 服务器的 dmfldr 模块，也就是 dmfldr 的服务器端，由其进行真正的数据载入工作。

dmfldr 客户端每次向服务器端发送一批数据，在服务器端插入数据的过程中，由于目的表上可能存在约束等原因，导致某些数据无法插入成功，此时服务器端会将这一批数据全部回滚，并将这批数据全部记为错误数据，但服务器端插入时的错误数据并不会记录到 BADFILE 中。

ERRORS 所统计的错误包含在数据转换和数据插入过程中所产生的数据错误，因此当服务器端插入数据记录的错误数据数加上客户端数据转换时的错误数据数超过 ERRORS 参数的指定值时，dmfldr 服务器会停止插入数据。

3.5 大字段数据处理

dmfldr 支持对 DM 数据库的大字段类型数据的载入和导出，DM 数据库支持的大字段数据类型有 TEXT、LONGVARCHAR、IMAGE、LONGVARBINARY、BLOB 以及 CLOB。

3.5.1 大字段数据的导出

当 dmfldr 工作在导出模式即 MODE 为 OUT 时，dmfldr 生成大字段对应的数据文件名由 LOB_FILE_NAME 指定，若未指定默认为 dmfldr.lob，文件存放于 LOB_DIRECTORY 指定的目录，如果未指定 LOB_DIRECTORY 则存放于指定的导出数据文件同一目录。

例如：

1） 建表 TEST

```
DROP TABLE TEST;
CREATE TABLE TEST(C1 INT,C2 BLOB,C3 CLOB);
```

2） 插入数据

```
INSERT INTO TEST VALUES(1,0XAB121032DE,'abcdefg');
INSERT INTO TEST VALUES(2,0XAB121032DE,'abcdefg');
COMMIT;
```

3） 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA
INFILE '/opt/data/test.txt'
INTO TABLE test
```

```
FIELDS '|'
(
C1,
C2,
C3
)
```

4) 使用 dmfldr 进行导出数据

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
LOB_DIRECTORY=\'/opt/data/\' mode=\'out\'
```

在这个例子中，指定了 LOB_DIRECTORY，而没有指定 LOB_FILE_NAME，导出的大字段数据文件将存放在 LOB_DIRECTORY 指定的/opt/data 目录，文件名为 dmfldr.lob。

3.5.2 DIRECT 为 TRUE 时大字段数据的载入

当 MODE 为 IN 且 DIRECT 为 TRUE 时，此时数据载入若涉及到大字段对象，需要用户指定大字段数据文件。若 CLIENT_LOB 为 TRUE，LOB_DIRECTORY 应指定大字段数据文件所在的客户端本地目录；若 CLIENT_LOB 为 FALSE，用户必须先把相关文件传送到 DM 服务器所在主库，然后使用 LOB_DIRECTORY 指明存放目录

大字段数据文件在数据文件中指定，可以是任意格式的文件。在数据文件中，大字段以“文件名：起始偏移：长度”的形式记录在数据文件中。指定的文件名无效时，dmfldr 会报错，装载失败。对于 CLOB 类型字段，当指定的偏移、长度范围内带有不完整字符时，dmfldr 将装载失败。

例如：

1) 建表 TEST

```
DROP TABLE TEST;
CREATE TABLE TEST(C1 INT,C2 BLOB,C3 CLOB);
```

2) 编辑数据文件 test.txt，存放路径为 DM 服务器所在主库的 /opt/data/test.txt，文件内容如下

```
1|testblob.txt:1:10|testclob.txt:1:10
```

```
2|testblob.txt:10:20|testclob.txt:10:20

3|testblob.txt:20:30|testclob.txt:20:30
```

其中，testblob.txt、testclob.txt 为文本文件，长度大于 30 字节，存放路径为/opt/data。

3) 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA

INFILE '/opt/data/test.txt'

INTO TABLE test

FIELDS '|'

(

C1,

C2,

C3

)
```

4) 使用 dmfldr 进行导入数据

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'

LOB_DIRECTORY=\'/opt/data/\'
```

3.5.3 DIRECT 为 FALSE 时大字段数据的载入

当 MODE 为 IN 且 DIRECT 为 FALSE 时，数据文件中大字段列数据即字段内容。BLOB_TYPE 参数指定 BLOB 列内容为十六进制或者字符串：

- BLOB_TYPE 为 HEX_CHAR 时，数据文件中 BLOB 列当作为十六进制内容；
- BLOB_TYPE 为 HEX 时，数据文件中 BLOB 列为字符串形式内容，导入后会转换为十六进制。

BLOB_TYPE 参数只对 DIRECT 为 FALSE 时有效，默认为 HEX_CHAR。

例 1：

1) 建表 TEST

```
DROP TABLE TEST;

CREATE TABLE TEST(C1 INT,C2 BLOB,C3 CLOB);
```

2) 编辑数据文件 test.txt，存放路径为/opt/data/test.txt，文件内容如下

```
1|0x12d3c8a7|abcdefg
2|0x12a4cbac|hijlkmn
3|0x22d3c8b3|adefhjd
```

3) 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA
INFILE '/opt/data/test.txt'
INTO TABLE test
FIELDS '|'
(
C1,
C2,
C3
)
```

4) 使用 dmfldr 进行导入数据，BLOB_TYPE 为 HEX_CHAR

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
direct=false blob_type=\'hex_char\'
```

5) 查询表数据

```
SELECT * FROM TEST;
行号       C1          C2         C3
---------- ----------- ---------- -------
1          1           0x12D3C8A7 abcdefg
2          2           0x12A4CBAC hijlkmn
3          3           0x22D3C8B3 adefhjd
```

例 2：

步骤 1)、2)、3)同例 1

4) 使用 dmfldr 进行导入数据，BLOB_TYPE 为 HEX

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
direct=false blob_type=\'hex\'
```

5) 查询表数据

```
select * from test;

行号        C1          C2                      C3

---------- ----------- ---------------------- -------

1           1           0x30783132643363386137 abcdefg

2           2           0x3078313261346263616163 hijlkmn

3           3           0x30783232643363386233 adefhjd
```

3.6 日志文件及日志信息

dmfldr 的日志文件路径由 LOG 参数设置，默认日志文件名为 fldr.log。文件记录了装载过程中的装载信息和错误信息以及统计信息。用户也可以通过设置控制文件中的 OPTION 选项来指定日志路径。如果参数及 OPTION 中同时指定了日志路径则其将以 OPTION 中指定的路径为最终路径。

例如：

1） 建表 TEST

```
DROP TABLE TEST;

CREATE TABLE TEST(C1 INT,C2 INT,C3 DATE);
```

2） 编辑数据文件 test.txt，存放路径为/opt/data/test.txt，文件内容如下

```
1|1 2015-11-06

2|2 2015-11-05

3|3 2015-11_04
```

3） 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA

INFILE '/opt/data/test.txt'

INTO TABLE test

FIELDS '|'

(

C1,

C2 TERMINATED BY ' ',

C3 DATE FORMAT 'yyyy-mm-dd'
```

```
)
```

4) 使用 dmfldr 进行数据载入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
log=\'/opt/data/test.log\'
```

5) 查看日志文件/opt/data/test.log，其内容如下

```
dmfldr: 2015-11-20 14:48:41  dmfldr:
Copyright (c) 2011, 2015, Dameng.  All rights reserved.
控制文件：
加载行数:全部
每次提交服务器行数:50000
跳过行数:0
允许错误数:100
是否直接加载:Yes
是否插入自增列:No
数据是否已按照聚集索引排序:No
字符集:GBK
dmfldr: 2015-11-20 14:48:41
数据文件共 1 个:
/opt/data/test.txt
错误文件:fldr.bad
目标表:TEST
列名     终止     包装数据类型
C1       |         CHARACTER
C2     WHT         CHARACTER
C3       |         yyyy-mm-dd
dmfldr: 2015-11-20 14:48:41
目标表:TEST
3 行加载成功。
由于数据格式错误,0 行 丢弃。
由于数据错误,0 行 没有加载。
```

```
跳过的逻辑记录总数:0
读取的逻辑记录总数:3
拒绝的逻辑记录总数:0
用时:40.831(ms)
```

3.7 自增列装载

自增列是比较特殊的列，为了保证数据库中自增列列值的正确性，用户在进行数据载入时需要特别注意。dmfldr 提供了 SET_IDENTITY 参数对数据载入时自增列的处理进行设置。

如果指定 SET_IDENTITY 选项值为 TRUE，则 dmfldr 将把从数据文件中读取的自增列值作为目标值插入数据库表中，用户应当保证每一行的自增列的值符合自增列的规则，否则将造成数据混乱；如果 SET_IDENTITY 选项值设置为 FALSE，则 dmfldr 将忽略数据文件中对应自增列的值，服务器将根据自增列定义和表中已有数据自动生成自增列的值插入每一行的对应列。SET_IDENTITY 默认为 FALSE。

用户也可以通过设置控制文件中的 OPTION 选项来设置 SET_IDENTITY 的值，参数值的优先选择顺序为先 OPTION 选项，后参数选项。

此参数为可选参数，作用于 MODE 为 IN 的情况下，当 MODE 为 OUT 时无效。

例 1：

1) 建表 TEST，并插入两行数据

```
DROP TABLE TEST;
CREATE TABLE TEST(C1 INT IDENTITY(1,1),C2 VARCHAR);
INSERT INTO TEST(C2) VALUES('AAA');
INSERT INTO TEST(C2) VALUES('BBB');
COMMIT;
```

此时表 TEST 中的数据为：

```
行号        C1          C2
---------- ----------- ---
1           1           AAA
2           2           BBB
```

2) 编辑数据文件 test.txt，存放路径为/opt/data/test.txt，文件内容如下

```
2|aaa
3|bbb
4|ccc
```

3) 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA
INFILE '/opt/data/test.txt'
INTO TABLE test
FIELDS '|'
(
C1,
C2
)
```

4) 使用 dmfldr 进行数据载入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
set_identity=false
```

5) 查看表 TEST 的数据

```
SELECT * FROM TEST;
行号        C1          C2
---------- ----------- ---
1           1           AAA
2           2           BBB
3           3           aaa
4           4           bbb
5           5           ccc
```

在这个例子中，表 TEST 中已有两行数据，由于 SET_IDENTITY 置为 FALSE，因此在数据载入时 dmfldr 根据 C1 列的定义和表中已有数据，为 C1 列重新插入合适的值。

我们再看看如果将 SET_IDENTITY 置为 TURE 结果会怎样。

例 2：

重复例 1 的 1)、2)、3）步骤

4） 使用 dmfldr 进行数据载入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
set_identity=true
```

5） 查看表 TEST 的数据

```
SELECT * FROM TEST;
行号        C1          C2
---------- ----------- ---
1           1           AAA
2           2           BBB
3           2           aaa
4           3           bbb
5           4           ccc
```

3.8 数据排序

SORTED 参数用来设置数据是否已经按照聚集索引排序，默认为 FALSE。

如果设置为 TRUE，则用户必须保证数据已按照聚集索引排序完成，并且如果表中存在数据，需要插入的数据索引值要比表中数据的索引值大，服务器在做插入操作时顺序进行插入。若数据并未按照索引排序，则 dmfldr 会报错，装载失败。

如果设置为 FALSE，则服务器对于每条记录进行定位插入。

用户也可以通过设置控制文件中的 OPTION 选项来设置 SORTED 的值。SORTED 参数值的优先选择顺序为 OPTION 选项，参数选项。此参数为可选参数，作用于 MODE 为 IN 且 DIRECT 为 TRUE 的情况下，对于其他情况此参数无效。

在数据量大，并且确定数据已按照聚集索引排序完成的情况下，将 SORTED 参数设置为 TRUE，可以提升装载性能。

例如：

1） 建表 TEST

```
DROP TABLE TEST;
CREATE TABLE TEST(C1 INT CLUSTER PRIMARY KEY,C2 VARCHAR);
```

2） 编辑数据文件 test.txt，存放路径为/opt/data/test.txt，文件内容如下

```
2|aaa
3|bbb
4|ccc
5|ddd
1|zzz
```

3) 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA
INFILE '/opt/data/test.txt'
INTO TABLE test
FIELDS '|'
(
C1,
C2
)
```

4) 使用 dmfldr 进行数据载入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
sorted=true
```

由于本例中数据文件中的数据并没有按照 C1 列排序，dmfldr 将会报错。

```
不能使用 NOSORT 选项，数据非有序
```

3.9 空值处理

dmfldr 通过设置 NULL_MODE 参数来处理空值。

- 设置为 TRUE，载入时 NULL 字符串处理为 NULL，载出时空值处理为 NULL 字符串
- 设置为 FALSE，载入时 NULL 字符串处理为字符串，载出时空值处理为空串

例 1：

1) 建表 TEST

```
DROP TABLE TEST;
CREATE TABLE TEST(C1 INT,C2 VARCHAR);
```

2) 编辑数据文件 test.txt，存放路径为/opt/data/test.txt，文件内容如下

```
1|aaa
2|NULL
3|null
```

3) 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA
INFILE '/opt/data/test.txt'
INTO TABLE test
FIELDS '|'
(
C1,
C2
)
```

4) 使用 dmfldr 进行数据载入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
null_mode=true
```

5) 查看表 TEST 的数据，数据文件中 C2 列的“NULL”和“null”字符串都被处理为空值

```
SELECT C1, IFNULL(C2, 'NULL VALUE') FROM TEST;
行号       C1          IFNULL(C2,'NULLVALUE')
---------- ----------- ----------------------
1          1           aaa
2          2           NULL VALUE
3          3           NULL VALUE
```

例 2：

步骤 1)、2)、3)与例 1 相同

4) 使用 dmfldr 进行数据载入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
null_mode=false
```

5) 查看表 TEST 的数据，数据文件中 C2 列的“NULL”和“null”字符串直接被作为字符串值插入表中

```
SELECT C1, IFNULL(C2, 'NULL VALUE') FROM TEST;

行号        C1          IFNULL(C2,'NULLVALUE')

---------- ----------- ----------------------

1           1           aaa

2           2           NULL

3           3           null
```

3.10 类类型装载

dmfldr 支持对 CLASS 类型数据的装载。CLASS 类型装载和 LOB 类型一样，载入时需要指定 LOB_DIRECTORY，另外需要指定 ENABLE_CLASS_TYPE 为 TRUE；导出时，默认导出目录和数据文件所在目录一致，导出类类型形成的大字段文件默认为 dmfldr.lob，导出目录和大字段文件可通过 LOB_DIRECTORY 和 LOB_FILE_NAME 参数设置。



注意：CLASS 类型大字段数据文件无法通过手动创建，只有从表中导出到大字段文件中，并使用数据文件 CLASS 字段对应的偏移、字长进行导入。

例如：

1) 建表 TEST

数据准备，创建类 mycls 的类头与类体，之后创建表 TEST，其 C2 列类型为 mycls 类类型，并向 TEST 中插入两行数据。

```
--类头创建

CREATE CLASS mycls

AS

TYPE rec_type IS RECORD (c1 INT, c2 INT);      --类型声明

id     INT;                         --成员变量

r      rec_type;                    --成员变量

FUNCTION f1(a INT, b INT) RETURN rec_type;     --成员函数
```

```
FUNCTION mycls(id INT , r_c1 INT, r_c2 INT) RETURN mycls; --用户自定义构造函数
END;
/

--类体创建
CREATE OR REPLACE CLASS BODY mycls
AS
FUNCTION f1(a INT, b INT) RETURN rec_type
AS
BEGIN
       r.c1 = a;
       r.c2 = b;
       RETURN r;
END;
FUNCTION mycls(id INT, r_c1 INT, r_c2 INT) RETURN mycls
AS
BEGIN
       this.id = id;       --可以使用 this.来访问自身的成员
       r.c1 = r_c1;        --this 也可以省略
       r.c2 = r_c2;
       RETURN this;        --使用 return this 返回本对象
END;
END;
/

--建表 TEST
DROP TABLE TEST;
CREATE TABLE TEST(C1 INT,C2 mycls);
--插入数据：
INSERT INTO TEST VALUES(1,mycls(1,1,1));
```

```
INSERT INTO TEST VALUES(2,mycls(2,2,2));
COMMIT;
```

2) 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA
INFILE '/opt/data/test.txt'
INTO TABLE test
FIELDS '|'
(
C1,
C2
)
```

3) 使用 dmfldr 进行数据导出

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
mode=\'out\' lob_directory=\'/opt/data\'
```

4) 在指定的 LOB_DIRECTORY 目录会生成大字段文件 dmfldr.lob，在指定的数据文件路径生成的数据文件内容如下：

```
1|dmfldr.lob:0:70
2|dmfldr.lob:70:70
```

5) 创建一张新表 TEST2，表的两列都为 mycls 类类型

```
CREATE TABLE TEST2(C1 MYCLS,C2 MYCLS);
```

6) 编辑数据文件 test2.txt，存放路径为/opt/data/test2.txt，文件内容如下：

```
dmfldr.lob:70:70|dmfldr.lob:0:70
dmfldr.lob:0:70|dmfldr.lob:70:70
```

7) 编辑控制文件 test2.ctrl，存放路径为/opt/data/test2.ctrl，内容如下：

```
LOAD DATA
INFILE '/opt/data/test2.txt'
INTO TABLE test2
FIELDS '|'
(
```

```
C1,
C2
)
```

8) 使用 dmfldr 进行数据导入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test2.ctrl\'
 lob_directory=\'/opt/data/\' enable_class_type=true
```

9) 查看表 TEST2 的数据

```
行号       C1                            C2
---------- ----------------------------- -----------------------------
1          SYSDBA.MYCLS(2,REC_TYPE(2,2)) SYSDBA.MYCLS(1,REC_TYPE(1,1))
2          SYSDBA.MYCLS(1,REC_TYPE(1,1)) SYSDBA.MYCLS(2,REC_TYPE(2,2))
```

3.11 个性化设置

用户通过设置 dmfldr 的 SKIP、LOAD、ROWS 参数，可以根据自己的需求调整装载的起始行、装载最大行数以及每次提交的行数。

SKIP 参数用来设置跳过数据文件起始的逻辑行数，整形数值。默认的跳过起始行数为 0 行。如果用户指定了多个文件，且起始文件中的行数不足 SKIP 所指定的行数，则 dmfldr 工具会扫描下一个文件直至累加的行数等于 SKIP 所设置的行数或者所有文件都已扫描结束。

LOAD 参数用来设置装载的最大行数，整形数值。默认的最大装载行数为数据文件中的所有行数。LOAD 指定的值是包括 SKIP 指定的跳过的行数。

ROWS 参数用来设置每次提交的行数，整形数值。默认的提交行数为 50000 行。提交行数的值表示提交到服务器的行数，并不一定代表按照数据文件中的数据顺序的行数。用户可以根据实际情况调整每次提交的行数，以达到性能的最佳点。ROWS 参数作用于 MODE 为 IN 的情况下，当 MODE 为 OUT 时无效。

例如：

1) 建表 TEST

```
DROP TABLE TEST;
CREATE TABLE TEST(C1 INT, C2 VARCHAR);
```

2) 编辑数据文件 test.txt，存放路径为/opt/data/test.txt，文件内容如下

```
1|aaa
2|bbb
3|ccc
4|ddd
5|eee
6|fff
7|ggg
8|hhh
9|iii
10|jjj
```

3) 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA
INFILE '/opt/data/test.txt'
INTO TABLE test
FIELDS '|'
(
C1,
C2
)
```

4) 使用 dmfldr 进行数据载入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
skip=3 load=5
```

5) 查看表 TEST 的数据，载入时跳过了数据文件的前 3 行数据，且只载入了 5 条数据。

```
SELECT * FROM TEST;
行号        C1          C2
---------- ----------- ---
1           4           ddd
2           5           eee
```

```
3          6          fff
4          7          ggg
5          8          hhh
```

3.12 主备切换时的数据继续载入

在 DM7 数据守护主备环境下，如果 dmfldr 数据载入中发生主备切换，dmfldr 支持主备切换完成后自动继续装载数据，并且能保持数据正确。

要使用此项功能，USERID 参数需要使用主备服务名方式进行配置，如：

```
dmfldr USERID=SYSDBA/SYSDBA@dw CONTROL='c:\fldr.ctl'
```

同时在 dm_svc.conf 配置文件中配置主备服务名，如：

```
dw=(192.168.0.101:5236, 192.168.0.102:5236)
```

目前主备切换时的数据继续载入功能还存在以下功能限制：

- 目前仅支持单机的主备，不支持 MPP 主备。

  但是，若在 MPP 主备环境中使用 MPP_CLIENT=FALSE，等同于单机，也是支持的。

- 目前不支持分区表装载

3.13 MPP 本地分发

MPP_CLIENT 参数用来设置在 DM MPP 环境下使用 dmfldr 进行数据装载时的数据分发方式。

当 DM 数据库服务器环境为单站点时此参数无效。当服务器环境为 MPP 环境时，若 MPP_CLIENT 为 TRUE，dmfldr 采用客户端分发模式；若 MPP_CLIENT 为 FALSE，则采用本地分发模式。

客户端分发模式下，数据在 dmfldr 客户端分发好后直接往指定站点发送数据。

本地分发模式下，导入时，dmfldr 直接将数据发送到连接的站点，数据最终在连接的站点；导出时，dmfldr 只导出连接站点的数据。

MPP 环境下要配置 dmmal.ini 文件中的 MAL_INST_HOST 和 MAL_INST_PORT 参数。

例如：

当前 DM 数据库服务器环境为 MPP 环境，两个节点，SEQNO 号分别为 0、1。使用 dmfldr 采用本地分发方式进行数据装载，连接的节点为 SEQNO 为 0 的节点。

1) 建表 TEST

```
DROP TABLE TEST;
CREATE TABLE TEST(C1 INT, C2 INT);
```

2) 编辑数据文件 test.txt，存放路径为/opt/data/test.txt，文件内容如下

```
1|1
2|2
3|3
4|4
5|5
6|6
7|7
8|8
9|9
10|10
```

3) 编辑控制文件 test.ctrl，存放路径为/opt/data/test.ctrl，内容如下：

```
LOAD DATA
INFILE '/opt/data/test.txt'
INTO TABLE test
FIELDS '|'
(
C1,
C2
)
```

4) 使用 dmfldr 进行数据载入

```
./dmfldr userid=SYSDBA/SYSDBA@localhost:5236 control=\'/opt/data/test.ctrl\'
mpp_client=false
```

5) 查询 SEQNO 为 0 的节点上 TEST 表的数据

```
SELECT * FROM TEST WHERE SF_GET_EP_SEQNO(ROWID)=0;
```

```
行号        C1          C2
---------- ----------- -----------
1           1           1
2           2           2
3           3           3
4           4           4
5           5           5
6           6           6
7           7           7
8           8           8
9           9           9
10          10          10
```

6) 查询 SEQNO 为 1 的节点上 TEST 表的数据

```
SELECT * FROM TEST WHERE SF_GET_EP_SEQNO(ROWID)=0;
未选定行
```

3.14 提升 dmfldr 性能

用户在使用 dmfldr 时根据系统和数据的具体情况对一些参数进行调整，可以获得更好的性能，这些参数包括：

- **BUFFER_NODE_SIZE**

BUFFER_NODE_SIZE 设置读取文件缓冲区页大小，值越大，缓冲区的页越大，每次读取的数据就越多，每次发送到服务器的数据也就越多，效率越高。但其大小受 dmfldr 客户端内存大小限制。

- **READ_ROWS**

在某些情况下，BUFFER_NODE_SIZE 读入的数据行数很大，而后续操作处理不了这么大的行数，此时可以用 READ_ROWS 来限制处理的行数。dmfldr 取 READ_ROWS 和 BUFFER_NODE_SIZE 中较小的值作为一次处理的行数。

- **SEND_NODE_NUMBER**

指定 dmfldr 在数据载入时发送节点的个数，默认由系统计算一个初始值。若在数据载入时发现发送节点不够用，系统会动态增加分配。在系统内存足够的情况下，可以适当设大 SEND_NODE_NUMBER 值，提升 dmfldr 载入性能。

- **TASK_THREAD_NUMBER**

指定 dmfldr 在数据载入时处理用户数据的线程数目。默认情况下，dmfldr 将该参数值设为系统 CPU 的个数，但当 CPU 个数大于 8 时，默认值都被置为 8。在 dmfldr 客户端所在机器 CPU 大于 8 环境中，提高 TASK_THREAD_NUMBER 值可以提升 dmfldr 装载性能。

- **BLDR_NUM**

水平分区表装载时，指定服务器 BLDR 的最大个数，默认为 100。

服务器的 BLDR 保存水平分区子表相关信息，BLDR_NUM 的设置也就指定了服务器能同时载入的水平分区子表的个数。若 BLDR_NUM 设置太大，当水平分区子表数过多时，可能会导致服务器内存不足。当载入时实际需要的 BLDR 个数超出 BLDR_NUM 设置时，会淘汰指定子表的 BLDR，并替换为新的子表 BLDR。

- **BDTA_SIZE**

BDTA（Batch DaTA）的大小，默认为 5000。

BDTA 代表 DM7 数据库批量数据处理机制中一个批量，在内存、CPU 允许的条件下，增大 BDTA_SIZE 能加快装载速度；在网络是装载性能瓶颈时，增大 BADT_SIZE 影响不大。

- **INDEX_OPTION**

索引的设置选项，默认为 1。INDEX_OPTION 的可选项有 1 和 2。

1 代表服务器装载数据时先不刷新二级索引，而是将新数据按照索引预先排序，在装载完成后，再将排好序的数据插入索引。如果在数据载入前，目标表中已有较多数据，建议 INDEX_OPTION 置为 1。

2 代表服务器在快速装载过程中不刷新二级索引数据，只在装载完成时重建所有二级索引。如果在数据载入前，目标表中没有数据或数据量较小，建议 INDEX_OPTION 置为 2。

3.15 dmfldr 使用限制

dmfldr 的使用存在以下一些限制：

- 不支持向临时表、外部表装载数据

- 不支持向系统表装载数据
- 不支持向带有位图索引的表装载数据
- 不支持向带有函数索引的表装载数据
- 不支持向带有全文索引的表装载数据
- 不支持单独对表垂直分区子表装载数据
- 不支持向 DCP 代理装载数据
- dmfldr 装载时，对约束进行检查，对各种约束的处理机制如下表所示

表 3.1 dmfldr 的约束检查机制

约束	数据不满足时	数据插入与否	约束是否有效
非空约束(NOT NULL)	报错	不插入	有效
聚集索引(CLUSTER PRIMARY KEY)	报错	不插入	有效
唯一约束(UNIQUE， PRIMARY KEY)	报错	插入	失效
引用约束(FOREIGN KEY)	不报错	插入	有效
CHECK 约束(CHECK)	不报错	插入	有效